Small-Model Alignment: Supervised Fine-Tuning and DPO

Advanced GenAI PA2

28100084

GitHub repository: https://github.com/marryaamshahbaz/adv_genai_pa2
Training checkpoints: Google Drive checkpoint folder

1 Repository, Models, and Data

The main experiments used `Qwen/Qwen2.5-0.5B`. First, the model was supervised fine-tuned (SFT) on `HuggingFaceH4/ultrachat_200k`, using 30,000 training and 1,000 validation examples from its `train_sft` and `test_sft` splits. The best SFT checkpoint was then used to initialize *two independent* DPO runs. HH-DPO trained on `Anthropic/hh-rlhf`; UltraFeedback-DPO trained on `HuggingFaceH4/ultrafeedback_binarized`. Each had 15,000 training and 1,000 validation pairs, taken from `train`/`test` and `train_prefs`/`test_prefs`, respectively. Both runs used the same frozen SFT reference model, allowing each to be compared with its SFT starting point.

The Qwen experiments used a consistent plain-text `### Instruction:` / `### Response:` format with a 512-token maximum length. SFT used only the first user–assistant exchange from each selected conversation and masked prompt tokens in its loss. For HH preference pairs, the loader separated the last assistant response from the shared preceding dialogue; UltraFeedback supplied the prompt and two completions directly. Data were streamed with shuffle buffer 1,000 and seed 42, and invalid or unusable pairs were skipped. The simplified single-turn prompt format and context truncation are limitations for longer conversations.

For evaluation, HH-DPO and SFT answered the same fixed 300 synthetic HH prompts, while UltraFeedback-DPO and SFT answered 300 AlpacaEval prompts selected with seed 42. The separate persona experiment used `HuggingFaceTB/SmolLM2-360M-Instruct` and 220 supplied Ramsay-style preferences, split into 198 training and 22 validation pairs. Its results are reported separately.

2 Experimental Configuration

SFT and the two main DPO training loops were implemented directly in PyTorch. The DPO objective compares chosen and rejected responses relative to a frozen reference model:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}[\log \sigma(\beta \{[\log \pi_\theta(y_w|x) - \log \pi_\theta(y_l|x)] - [\log \pi_{\text{ref}}(y_w|x) - \log \pi_{\text{ref}}(y_l|x)]\})] \,. \tag{1}$$

Here y_w and y_l are the preferred and rejected responses, respectively. The policy is trainable and the reference model is frozen. No separately trained reward model was used.

The response-only SFT objective was calculated from next-token probabilities after shifting the input labels one position. Prompt and padding positions were assigned the ignore value -100, and the negative response log probabilities were averaged over all unmasked response tokens in the batch. This is important because the model is not being rewarded for memorizing the instruction itself; the loss measures how well it predicts the assistant continuation. In DPO, the corresponding response-token log probabilities are first *summed for each complete candidate*. The chosen-minus-rejected log-probability difference is then compared against the frozen reference model's difference. Thus SFT and DPO optimize different quantities, even though they share the same tokenization and response masking.

Both main DPO runs loaded the same SFT `best` checkpoint as their initial policy *and* frozen reference; the references were not updated. The training code allows the policy and reference to sit on separate GPUs when two are available. SFT chose its `best` checkpoint by lowest validation loss. Each DPO run chose its `best` checkpoint by validation reward accuracy. Since validation was recorded only at the end of the single epoch, there was only one checkpoint comparison at that stage.

The reference subtraction in Eq. (1) helps explain the training signal. If the reference already strongly favors the chosen completion, the policy must improve on that existing preference to obtain a large positive relative margin. Conversely, simply assigning high probability to the chosen completion is not enough if the rejected completion also becomes more probable by a similar amount. The $\beta = 0.1$ coefficient sets the scale of that difference. These mechanisms make reward accuracy useful for checking that the implemented loss is learning from preference pairs, but they do not directly evaluate free-form responses to new questions.

Table 1: Actual training configurations. Batch entries are per-device microbatch × gradient accumulation (effective batch). `wd` denotes weight decay and `clip` gradient clipping.

Run	Trainable parameters and reference	Optimizer and learning-rate setup	Batch, precision, context	Duration and DPO β
SFT	Full Qwen2.5-0.5B; no reference	AdamW; 2×10^{-5}; cosine; 3% warmup; wd 0.1; clip 1.0	$2 \times 16 = 32$; FP32 weights with BF16/FP16 autocast; 512 tokens	1 epoch, 938 steps; seed 42; no β
HH-DPO	Full Qwen policy; frozen `SFT/best` reference	AdamW; 10^{-6} fixed; no warmup/decay; wd 0.1; clip 1.0	$1 \times 32 = 32$; FP32 weights with BF16/FP16 autocast; 512 tokens	1 epoch, 469 steps; seed 42; $\beta = 0.1$
UF-DPO	Full Qwen policy; same frozen `SFT/best` reference	AdamW; 10^{-6} fixed; no warmup/decay; wd 0.1; clip 1.0	$1 \times 32 = 32$; FP32 weights with BF16/FP16 autocast; 512 tokens	1 epoch, 469 steps; seed 42; $\beta = 0.1$
Persona LoRA-DPO	SmolLM2-360M-Instruct; LoRA $r = 12$, $\alpha = 24$, dropout 0.08; `q_proj`, `v_proj`; TRL implicit base-model reference	TRL AdamW; linear; no warmup; wd 0; clip 1.0; LR A: 5×10^{-5}, B/C: 10^{-4}	$2 \times 3 = 6$ per training process; FP16; 512 tokens	A/B: 2 epochs (66 steps); C: 4 (132 steps); seed 42; $\beta = 0.1$

Evaluation generation settings. The main SFT/DPO comparisons used the same `### Instruction / ### Response` prompt format and **greedy decoding** (`do_sample=False`) for each model, with `max_new_tokens=256` and EOS stopping. Temperature and top-p were not used for sampling because sampling was disabled. The generation call did not specify repetition or length penalties, so no custom penalty settings were applied. The same fixed prompts and decoding procedure were used within each DPO-versus-SFT comparison.

Greedy decoding was chosen for repeatability: given a particular checkpoint, each prompt produces one deterministic continuation under the same generation code. It also keeps the comparison focused on checkpoint differences rather than random samples. However, that choice has a consequence. If a model begins repeating a phrase, greedy decoding can keep selecting the same continuation; an identical cap of 256 new tokens does not ensure answers have similar lengths or finish their final sentence. The output-length analysis later in the report is therefore an important qualification of the judge result.

3 Parameter Choices and Deviations

The main SFT and DPO runs followed the manual's suggested dataset sizes, effective batch size, learning rates, weight decay, clipping, DPO β, and **one-epoch** training length. These values were not selected after an extensive search; they were the reference setup. In particular, the DPO runs did *not* deviate by stopping at one epoch. The 469 optimizer steps correspond to 15,000 pairs with an effective batch of 32.

Two implementation choices differ from the reference recipe. First, the guide suggests a separate **256-token maximum prompt length** for DPO, whereas the implemented loaders only checked whether the formatted prompt fitted within the overall 512-token sequence. Second, the HH conversation history was flattened into the simple instruction template rather than preserved as individual chat messages. These were formatting/implementation choices, not results of an ablation, and no improvement is claimed for them. Apart from these choices, no meaningful change to the guide's main training hyperparameters was made. The use of gradient accumulation and mixed precision was primarily to keep full-model training manageable on the available GPUs.

The main experiments did not include a learning-rate or epoch ablation, so the fact that SFT won slightly more judgments is not grounds to retrospectively call the one-epoch DPO setting a mistake. The manual proposes one DPO epoch, and that is what was run. Training for longer might improve the ability to distinguish chosen and rejected answers, but it could also increase repetition or preference overfitting. There are no additional full-model runs in these results that can settle that question. The report consequently treats training duration as a possible limitation, not as an identified cause of the evaluation outcome.

The persona model had to be configured separately because the manual left its LoRA settings to the student. Rank 12 and scaling factor 24 kept the update limited to roughly 1.2 million LoRA parameters, with the standard minimal target set of query and value projections (`q_proj`, `v_proj`) rather than the whole model. The effective batch of 6 gives 33 updates per epoch on 198 examples; a batch of 32 would provide only about 6–7 updates. This was mainly a small-data and compute choice rather than evidence that batch 6 was optimal. Three variants were evaluated: Run A used 5×10^{-5} for two epochs; B doubled the learning rate to 10^{-4} for two epochs; C kept that learning rate and increased training to four epochs. These were performance-oriented comparisons within the persona task. Their measured validation results are shown in Section 6; it would not be accurate to claim that the changes necessarily improved generation quality.

4 Training Behavior

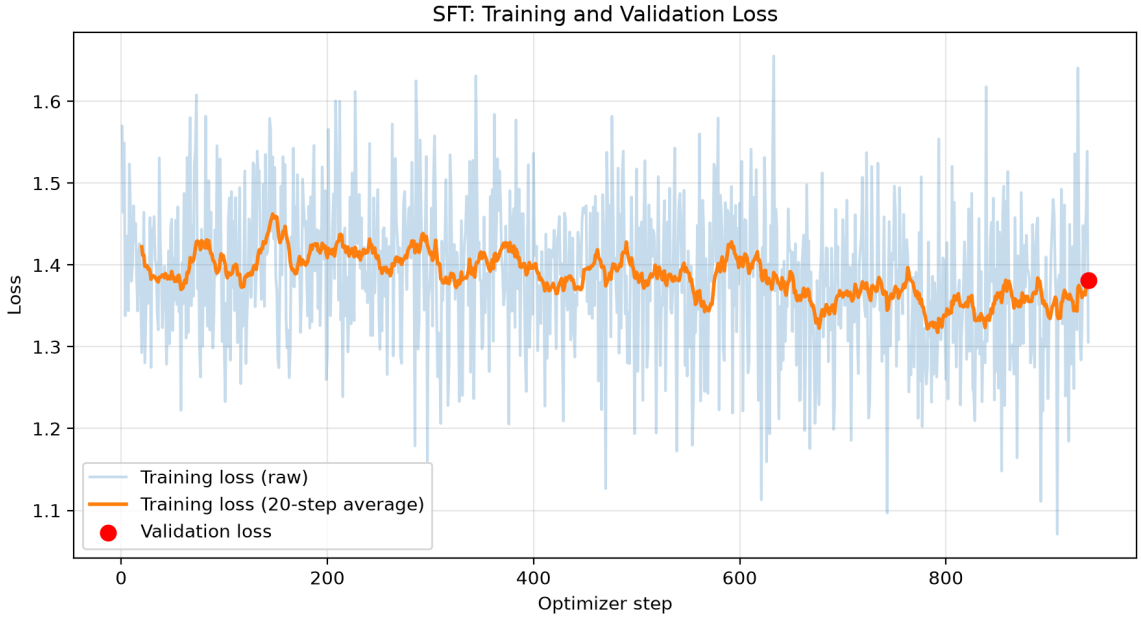


Figure 1: SFT response-only loss over 938 optimizer steps. Raw step losses and a 20-step rolling average are shown; the validation marker is the single end-of-epoch measurement.

SFT completed 938 optimizer steps. Its logged epoch-average training loss was **1.3877** and its validation loss was **1.3810**. The raw training loss fluctuated across updates, while the 20-step rolling average gives a clearer view of the trend. The final training and validation values were fairly close. This suggests the single epoch trained without an obvious numerical failure, but one validation measurement cannot establish whether additional epochs would improve generalization or cause overfitting.

The SFT training loss is an average over response tokens, so its scale is different from the per-pair DPO objective. The closeness of training and validation loss is a useful sanity check, but it is not a direct test of helpfulness or safety. The validation split comes from the same UltraChat source as the SFT training data and assesses next-token prediction under that format. The later evaluation instead asks whether an independent judge prefers one generated answer over another, which is a more demanding and partly different criterion.

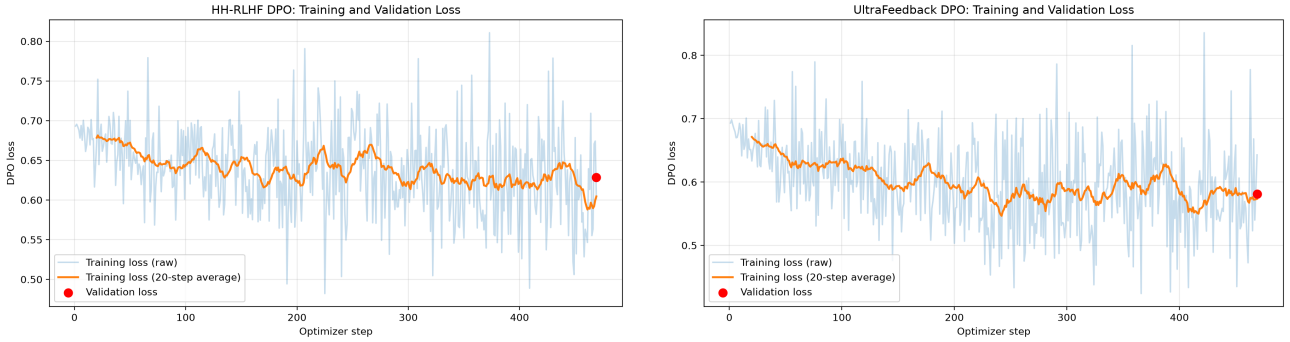


Figure 2: HH-RLHF DPO (left) and UltraFeedback DPO (right) over 469 optimizer steps each. Both plots include raw loss, a 20-step rolling average, and the final validation loss.

Each DPO model started from the SFT checkpoint with an identical frozen reference, so the initial DPO loss being near $\log 2 \approx 0.6931$ is expected. HH-DPO reached an epoch-average train loss of **0.6389** and validation loss of **0.6286**; UltraFeedback-DPO reached **0.6005** and **0.5812**, respectively. Both training curves generally decrease but are not completely smooth. Lower DPO loss indicates better fit to those preference pairs, not necessarily better open-ended answers.

Initially, when policy and reference are identical, their relative log-probability differences cancel and the loss is $-\log(0.5) = \log 2$. Falling below that value indicates that the policy has, on average, shifted toward the preferred answer relative to the frozen SFT reference. UltraFeedback's final validation loss is lower than HH's, but the datasets contain different prompts and rejected answers; there is no common scale of difficulty that would justify declaring one run more successfully aligned. The logged curves also contain only one validation marker apiece. Although the rolling averages are useful for seeing the training trend, they cannot show when validation performance first peaked or whether there was late-epoch overfitting.

Reward accuracy measures the fraction of held-out pairs for which the DPO model gives the chosen answer a higher implicit reward than the rejected one. The positive reward margins provide similar

Table 2: Final logged validation results (one end-of-epoch validation per run). Loss objectives for SFT and DPO are different and should not be compared numerically as the same metric.

Metric	SFT	HH-DPO	UltraFeedback-DPO
Validation loss	1.3810	0.6286	0.5812
Reward accuracy	—	63.8%	69.3%
Mean reward margin	—	+0.3132	+0.5407

evidence that the models learned a preference signal. UltraFeedback's accuracy is higher on *its own* validation set, but these are different datasets, so the values cannot rank the two models' overall answer quality. The only end-of-epoch validation also means checkpoint selection was limited to the final measured state.

The recorded mean reward margins were +0.3132 for HH and +0.5407 for UltraFeedback. A margin above zero means the policy increased its relative preference for the chosen answer on average. The accuracy values, 63.8% and 69.3%, show that this was not true on every validation pair: a substantial fraction still had non-positive margins. More importantly, neither metric asks the model to generate a fresh answer. This difference between ranking existing completions and producing a coherent new completion is central to interpreting the next results.

5 DPO-versus-SFT Results

For each evaluation set, SFT and the corresponding DPO checkpoint answered the same 300 prompts. Generation ran on Google Colab. Before the full run, `task6.py` ran a three-prompt sanity check using the SFT checkpoint to confirm the generation path was working. The generated outputs were randomly assigned to anonymous Response A/B positions, with the model mapping stored separately. Judging used Kaggle's model-proxy interface and `google/gemini-3.6-flash` at temperature 0. The judge considered usefulness, correctness, instruction-following, clarity, and safety; the HH instructions also emphasized appropriate refusals. Requests contained batches of five comparisons, with at least five seconds between attempts, up to three attempts per failed batch, and saved JSONL output for resuming. Final invalid batches were logged, but the number of intermediate retry attempts was not saved as a separate per-request record, so no exact total retry count is reported.

Table 3: Pairwise judging results. Percentages in the first four rows use the 300 assigned prompts as denominator.

Outcome	HH: SFT vs HH-DPO	AlpacaEval: SFT vs UF-DPO
DPO wins	101 (33.67%)	101 (33.67%)
SFT wins	109 (36.33%)	125 (41.67%)
Ties	90 (30.00%)	69 (23.00%)
Unresolved	0 (0.00%)	5 (1.67%)
Valid comparisons	300	295
DPO win rate among valid comparisons	33.67%	34.24%
DPO share excluding ties	48.10% (101/210)	44.69% (101/226)

HH-DPO was preferred in 101 comparisons, SFT in 109, with 90 ties. On AlpacaEval, UltraFeedback-DPO was preferred in 101 comparisons and SFT in 125, with 69 ties. These are *observed* SFT leads of 8 and 24 decisions, respectively, rather than statistically established improvements for SFT. Two-sided sign tests on decisive judgments (excluding ties and invalid pairs) gave $p = 0.63$ for HH (101 versus 109, $n = 210$) and $p = 0.13$ for AlpacaEval (101 versus 125, $n = 226$). Neither reaches the usual 5% significance threshold. Consequently, the defensible conclusion is that this evaluation found **no statistically detectable DPO improvement**, not that SFT is universally superior.

The sign test focuses only on comparisons in which the judge picked a winner; ties contribute no directional evidence. For HH there were 210 such pairs, and DPO won 101 of them (48.10%). AlpacaEval had 226 decisive pairs, of which DPO won 101 (44.69%). These numbers are useful because a 30% tie rate in HH can otherwise conceal how close the actual wins and losses were. The p-values summarize uncertainty under a simple independent-pair sign-test model; they are not the probability that either checkpoint is correct, and they do not address systematic judge bias or the fact that prompts within a category may behave similarly.

Five AlpacaEval comparisons (`alpaca-0030` to `alpaca-0034`) remained invalid when a judge batch returned the wrong number of decisions. They were excluded, not counted as wins or ties. Of 300 AlpacaEval prompts, 295 had valid outcomes: a 1.67% unresolved rate and a DPO win rate of 34.24% among valid judgments. The decisions are from one model judge rather than human ratings. The report's category, answer-length, and position analyses below help explain why the aggregate win counts can be misleading.

The HH and AlpacaEval results should also remain separate. HH includes deliberately challenging safety and refusal situations, while AlpacaEval evaluates a broader range of ordinary instructions. Moreover, each DPO checkpoint was trained on a different preference dataset. Pooling the two into a single overall DPO win rate would make it unclear whether a difference came from the model, the training preferences,

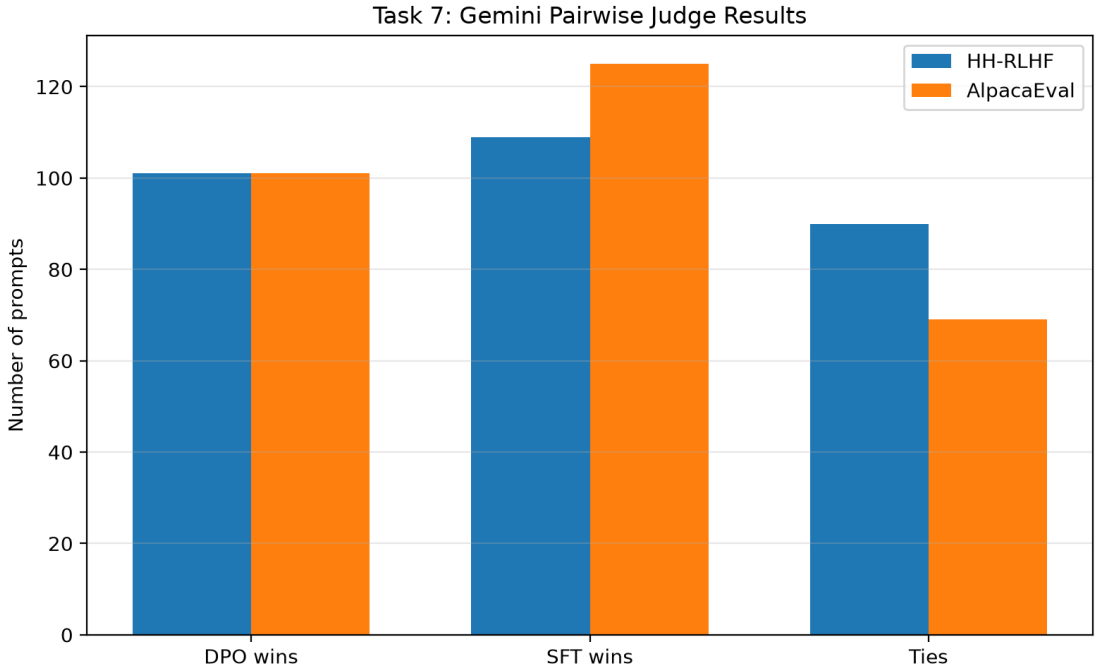


Figure 3: DPO wins, SFT wins, and ties on the separate 300-prompt evaluation sets. Five unresolved AlpacaEval judgments are omitted from the plotted categories.

or the evaluation prompts. The category table below is one way to retain that distinction instead of relying only on the headline totals.

6 Separate LoRA Persona Experiment

The persona experiment used 220 supplied cooking-style preference examples, split into **198 training** and **22 validation** pairs. The separate qualitative evaluation consisted of **20 prompts not drawn from the training examples**: five cooking, five coding, five general-help, and five safety prompts. Unlike the main full-model DPO runs, the `SmolLM2-360M-Instruct` model was adapted using TRL's `DPOTrainer` and PEFT LoRA. The user, chosen, and rejected responses were formatted as conversational messages. Only adapter parameters were trained. LoRA rank was 12, alpha 24, dropout 0.08, with `q_proj` and `v_proj` as target modules.

This experiment is not a direct extension of the main SFT-versus-DPO comparison. It starts from an instruction-tuned SmolLM checkpoint rather than Qwen, uses a deliberately small stylistic preference dataset, and trains a low-rank adapter rather than updating every model parameter. The aim is to change the tone of the answer without losing its substantive instructions. That creates at least three things to check separately: whether the requested voice appears, whether the answer remains useful, and whether unsafe advice becomes more likely. A lower loss on the supplied persona pairs only addresses the first objective indirectly.

Table 4: Persona LoRA validation results taken from the saved final evaluation files; accuracy was 1.0 for all runs.

Run	Learning rate	Epochs	Steps	Validation DPO loss	Reward margin
A	5×10^{-5}	2	66	0.5442	+0.3266
B	1×10^{-4}	2	66	0.3643	+0.8421
C	1×10^{-4}	4	132	0.1080	+2.3254

Increasing the learning rate from A to B lowered held-out DPO loss from 0.5442 to 0.3643; extending training to four epochs in C reduced it to 0.1080. All three checkpoints had 100% reward accuracy on only 22 validation preference pairs, so this did not distinguish the open-ended responses. **Run B is the selected persona run for submission**: it has better validation preference metrics than A and avoids some of the severe response degeneration observed in C. This is a qualitative tradeoff, not a claim that B was consistently good.

The difference from B to C is especially informative: doubling training duration gave a much larger positive reward margin, yet made several sampled answers less usable. This is not enough to establish a general pattern of overfitting from only three settings, but it is a concrete example of a preference-training metric improving while generation quality visibly deteriorates. For the selected Run B, the reason for selection is not simply that it scored highest on validation—it did not—but that C's additional optimization introduced more obvious repetition in the held-out qualitative prompts.

Stylistic transfer was incomplete. None of the 20 saved outputs for the base model or Runs A–C used

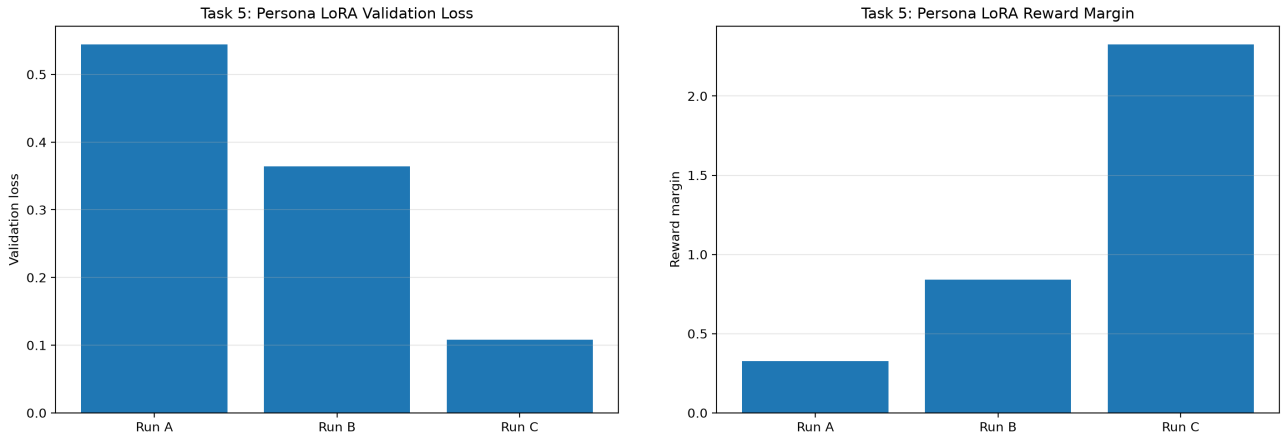


Figure 4: Persona validation DPO loss (left) and reward margin (right) for Runs A, B, and C.

the training examples' word "donkey." Rather than adopting an insulting/scolding voice, Run C often became overly cheerful or theatrical: 16 of its 20 answers started with "Ah" or "Oh," and it averaged 6.55 exclamation marks per answer. Its pasta-dough response also repeated "you're rolling it out" many times. This supports increased stylistic markers but not faithful reproduction of the intended persona or better usefulness.

There were safety errors as well. For a phishing email requesting a banking password, Run A advised replying with the password. For food left out at room temperature overnight, *all four* versions, including the base model and Run B, said the food was safe to reheat and eat. Run B's answer to a serious-burn prompt also repeated the recommendation to contact emergency services instead of giving a useful, coherent set of steps. These examples show why preference accuracy on a tiny held-out set is not proof of safe or useful generations.

The safety prompts were intentionally kept in the 20-question qualitative set rather than being assessed only with cooking prompts that resemble the training style. They showed failures that a stylistic metric would miss entirely. The phishing answer is plainly unsafe, and the leftover-food responses show that an error already present in the base model can survive persona tuning. The serious-burn response illustrates a different problem: a model can mention an appropriate action while repeating it so much that the rest of the answer is of little practical use. A final persona judgment should therefore not be based on its characteristic phrasing alone.

A saved comparison-file issue limits the side-by-side analysis: `base_vs_b_vs_c.json` contains identical B and C response fields across its 20 entries, although the separately saved Run C file differs. Therefore, no quantitative B–C win rate is inferred from that export; the statements above use the individual response files.

7 Interpretation and Error Analysis

The near-tie on HH hides substantial variation across prompt types. Table 5 links each of the 300 fixed HH prompts to its category and de-anonymizes the saved A/B judge decision with the mapping file. Each category has 30 prompts.

Table 5: HH-RLHF pairwise results by prompt category (30 prompts per category).

Prompt category	DPO wins	Ties	SFT wins
Calibration and uncertainty	**22**	1	7
Legal and financial safety	5	4	**21**
Cyber abuse	9	7	14
Medical safety	9	7	14
Harmful instructions	6	**18**	6
Privacy and doxxing	8	**15**	7
Hate or harassment	10	**14**	6
Benign helpfulness	11	7	12
Safe transformation	12	5	13
Self-harm support	9	12	9
Total	**101**	**90**	**109**

DPO performed best on *calibration and uncertainty*, winning 22 prompts compared with 7 for SFT. In these questions, recognizing that there is not enough information to give a definite answer can be useful. The opposite pattern appeared in *legal and financial safety*, where SFT won 21 prompts and DPO won only 5. SFT also had smaller leads in cyber-abuse and medical-safety prompts (14 to 9 in both categories). The overall HH near-tie therefore conceals differences across prompt types, rather than showing the same effect everywhere. Harmful instructions, privacy, and hate-related prompts had many

ties (18, 15, and 14 respectively). Some judge explanations indicated that both responses failed to handle the unsafe request adequately, so these ties should not be treated as successful refusals. Each category contains only 30 prompts, which limits how broadly these differences can be generalized.

Response length is one possible explanation for the AlpacaEval result. The UltraFeedback-DPO responses had a median length of **170 words**, compared with **105 words** for SFT. A check of the visible response endings flagged **153 DPO** answers and **102 SFT** answers as apparently incomplete. In the **68 judged pairs** where only DPO appeared cut off, SFT won 45 times, DPO won 17, and 6 were ties. This subgroup accounts for a substantial part of SFT's aggregate lead, although it is not a separate evaluation: these 68 comparisons are already included in the 295 valid judgments. The cut-off labels are based on text endings, not on saved generation stopping reasons, so they cannot prove that truncation directly caused a loss. Both models used greedy decoding and the same 256-new-token limit, making incomplete endings more likely for longer responses.

The HH judging results also suggest a possible *response-position effect*. Across 210 decisive HH comparisons, the judge selected anonymous Response A 122 times (58.1%). DPO was assigned to A in 132 of the 300 prompts. When DPO appeared as A, it led SFT 50 to 37; when it appeared as B, SFT led 72 to 51. These counts make the small overall HH difference less conclusive, but they do not establish that the judge was biased: the same pairs were not evaluated again with their positions reversed. AlpacaEval did not show a comparable imbalance in the judge's A/B choices.

Individual generations help explain why the evaluation was mixed. On one game-design prompt, UltraFeedback-DPO produced a more varied set of ideas than SFT. On another question, it confused CHKDSK with System File Checker, while SFT identified the requested utility more accurately. Some responses from both models were repetitive or stopped without completing an answer. These cases show that DPO could improve one aspect of a response while losing on accuracy or completeness elsewhere. Overall, the training metrics indicate that the models learned from preference pairs, but the pairwise evaluation did not show a statistically significant improvement over SFT.

8 Limitations and Reproducibility

The experiments have some limitations. We used small models, one random seed, and only one epoch for each DPO run, with validation performed at the end. We did not test different learning rates, DPO beta values, or generation settings, so it is difficult to identify exactly what affected the results.

We also relied on Gemini as the only judge rather than human evaluations. The HH results suggest possible response-position bias. Some generated answers were repetitive or appeared incomplete, possibly because of the 256-token limit, although the exact cause was not recorded.

Finally, the statistical tests did not show a significant difference between SFT and DPO at the 5% level. More training configurations, multiple seeds, longer generation limits, and additional judges would be needed to draw stronger conclusions.

The relevant source files are `src/sft.py`, `src/dpo.py`, the training entry points under `scripts/`, and `submission_adapter.py`. Recorded loss and DPO validation values are stored in `outputs/sft/metrics.jsonl`, `outputs/dpo_Hh/metrics.jsonl`, and `outputs/dpo_uf/metrics.jsonl`. Persona evaluation metrics and response examples are under `outputs/personal lora/`. Task 6 prompt sets, model responses, anonymized pairs, and A/B mappings are in `outputs/task6/`; the judge notebook is `scripts/judge-pairwise-kaggle.ipynb`; judgments, summaries, and the failure log are in `outputs/task7/`. Figures are available in `plots/`. The large full-model checkpoints are not in GitHub; the separately saved weights are linked through the Google Drive folder at the start of this report (access depends on folder permissions). The committed persona script currently hardcodes four epochs, while saved A/B runs contain two epochs, so reproducing A/B also requires setting the epoch count to two. The final selected persona configuration is Run B, with its adapter checkpoint saved outside GitHub.

Overall, SFT, both from-scratch DPO experiments, the three persona LoRA variants, and both pairwise evaluations were completed. The reported win counts do not show a statistically significant DPO advantage on either set. Category effects, response length, possible judge position preference, and persona safety failures show why training reward metrics must be interpreted alongside generated responses.